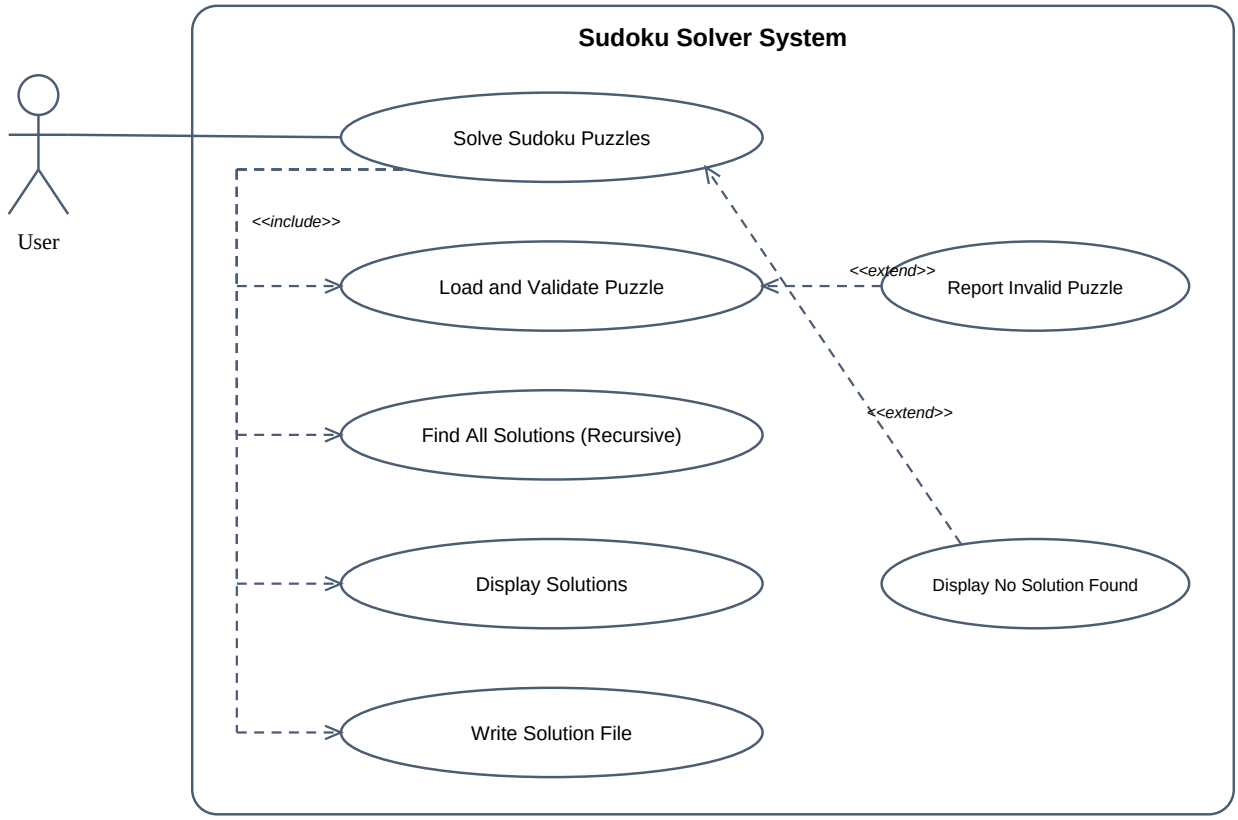
Sudoku Solver - UML Use Case Diagram

EECS 348 Assignment 5 | Actor: User | System: Sudoku Solver | Revised by: Nathaniel Lanter (based on the Assignment 4 diagram and code by Peter Lauman)



Main use case: Solve Sudoku Puzzles. **Includes:** Load and Validate Puzzle, Find All Solutions (Recursive), Display Solutions, Write Solution File. **Extensions:** Report Invalid Puzzle (extends Load and Validate Puzzle); Display No Solution Found (extends Solve Sudoku Puzzles).

Precondition: Assignment5.exe and puzzle1.txt - puzzle5.txt are in the working directory. **Postcondition:** Every valid solution is displayed and written to solutionN.txt, or the no-solution result is shown and written.

1. Identify the Use Case

Use case name: Solve Sudoku Puzzles.
Discovery: The User wants to know every valid completion of each of five Sudoku puzzles and to keep those answers in files. That goal is a single observable result for the User, so it is the main use case. Four sub-behaviors are always needed to reach it, so they are modeled as included use cases: Load and Validate Puzzle, Find All Solutions (Recursive), Display Solutions, and Write Solution File. Two behaviors happen only in special situations, so they are modeled as extensions: Report Invalid Puzzle (the file cannot be used) and Display No Solution Found (the puzzle has zero solutions).
Changes from Assignment 4: the new Write Solution File use case, an extension for invalid puzzles that also writes a result file, and a final "Solutions found: n" or "No solution found." line in each output file.

2. Brief Description

The User runs Assignment5.exe in the folder that holds puzzle1.txt through puzzle5.txt. For each puzzle the application loads and validates the grid, shows the starting grid on the console, uses recursive backtracking to find all valid solutions, prints them on the console, and writes them to the matching solutionN.txt file. If a puzzle has no solution, or cannot be used, the file contains the single line "No solution found."

3. Step-by-Step Flow of Events (Outline)

1. User launches Assignment5.exe.
2. System selects the next puzzle file (puzzle1.txt through puzzle5.txt).
3. System loads the 81 entries and validates the starting clues.
4. System displays the file name and the starting grid.
5. System chooses the empty square with the fewest legal candidates.
6. System tentatively places a candidate and searches recursively.
7. Whenever the grid is complete, System displays the solution and appends it to solutionN.txt.
8. System backtracks and continues until every possibility has been tried.
9. System writes the final line to solutionN.txt ("Solutions found: n" or "No solution found.") and closes the file.
10. System repeats for the remaining puzzles, then the program exits after the fifth.

4. Detailed Description

Use case: Solve Sudoku Puzzles

Primary actor: User (a student or a grader running the program from the Cycle servers or a lab machine).

Trigger: User launches the compiled executable (Assignment5.exe) in the folder containing the puzzle files.

Preconditions: Assignment5.exe exists; puzzle1.txt through puzzle5.txt are in the current working directory; the directory allows new files to be created and overwritten.

Input format: Each puzzle file has 81 whitespace-separated tokens in nine rows of nine. A token is a digit 1-9 or an underscore (_) for an empty square.

Main success scenario

1. User launches Assignment5.exe.
2. System builds the file names puzzleN.txt and solutionN.txt for the current puzzle number N (1 to 5) and creates a new SudokuSolver object.
3. System opens puzzleN.txt, reads exactly 81 tokens, converts underscores to empty squares, and rejects extra tokens.
4. System checks every given digit against its row, column, and 3-by-3 block to confirm there are no conflicting clues.
5. System prints "=== puzzleN.txt ===" and the original grid on the console.
6. System opens (creates or overwrites) solutionN.txt.
7. System scans every empty square, computes its legal candidates, and selects the square with the fewest candidates.
8. System assigns each candidate in turn and calls itself recursively on the smaller puzzle.
9. When no empty square remains, System increments the solution count, prints "Solution k:" and the grid on the console, and writes the same grid (nine rows, blank line after) to solutionN.txt in the input format.
10. System undoes each tentative assignment (backtracks) so that all other candidates and all other solutions are explored.
11. When the search ends with at least one solution, System prints "Total solutions: n" on the console and writes "Solutions found: n" as the last line of solutionN.txt.
12. System closes solutionN.txt and moves to the next puzzle. After puzzle 5 the program returns 0 and exits.

Alternate and exception flows

A1 - Invalid puzzle (extension: Report Invalid Puzzle). If the file cannot be opened, has fewer or more than 81 tokens, contains an unexpected token, or has conflicting clues, System prints "Unable to read puzzle or invalid starting grid.", writes the single line "No solution found." to solutionN.txt, and continues with the next puzzle.

A2 - Dead end. If an empty square has no legal candidates, that recursive branch returns without a solution and the caller tries its next candidate.

A3 - No solution (extension: Display No Solution Found). If the search finishes with zero solutions, System prints "No solution found" on the console and solutionN.txt contains only the single line "No solution found."

A4 - Multiple solutions. If more than one solution exists, System displays and writes all of them, not only the first, followed by "Solutions found: n".

A5 - Existing output file. If solutionN.txt already exists, it is overwritten so that it reflects only the current run.

Postconditions

Each of the five puzzles has a matching solutionN.txt. A file for a solvable puzzle holds every solution in the input format followed by "Solutions found: n". A file for an unsolvable or unusable puzzle holds exactly "No solution found." The console shows the original grid and the results for each readable puzzle.

Implementation details

The class SudokuSolver owns a 9-by-9 grid, a solution counter, and an output stream for the solution file. **load()** reads and validates the puzzle; **allowed()** checks row, column, and 3-by-3 block constraints; **chooseCell()** selects the empty square with the fewest candidates; **backtrack()** recursively enumerates all solutions and writes each to the console and the file; **print(std::ostream&)** formats the grid for the console or a file; **solve(solutionName)** opens the solution file, starts the search, and writes the final summary line. **main()** loops over the five puzzles and writes "No solution found." for any puzzle that fails to load.